

SPLUNK SOC LAB

Nmap Threat Detection & Risk Triage

Security Operations Center — Detection Engineering Project

Prepared by

Vijay S

SOC Analyst / Penetration Tester (Candidate)

reachvijays20@gmail.com

github.com/JacobVijay26 · linkedin.com/in/vijaycybersecurity

Splunk Enterprise 10.4.0 · Kali Linux · Metasploitable2

Isolated VMnet1 Host-Only Lab Environment

Table of Contents

3

3

3

3

4

4

5

6

6

7

8

9

9

1. Executive Summary

This project demonstrates an end-to-end Security Operations Center (SOC) workflow built in Splunk Enterprise. Output from a live penetration test against a Metasploitable2 target — specifically an Nmap scan revealing 23 open TCP ports — was ingested into Splunk and engineered into a functioning detection and triage capability.

Rather than simply storing scan data, the project parses raw events into structured fields, classifies each exposed service by risk severity, visualizes the results in a two-panel SOC dashboard, and configures a scheduled alert that fires when critical backdoor services are detected. The result is a compact but realistic demonstration of the analyst skill set: ingest, parse, classify, visualize, and alert.

OUTCOME A reusable Splunk pipeline that converts raw Nmap output into a severity-ranked SOC dashboard and an automated detection alert for instant-root backdoor services.

2. Lab Environment

All work was performed inside an isolated, host-only virtual network to ensure no exploit traffic could reach external systems.

Component	Detail
SIEM Platform	Splunk Enterprise 10.4.0 (Windows host)
Attacker	Kali Linux — scan origin, host label "kali-attacker"
Target	Metasploitable2 — intentionally vulnerable Linux host
Network	Isolated VMnet1 host-only (no internet egress)
Data Source	Nmap scan output (scan1.txt), sourcetype nmap_scan

3. Methodology

3.1 Data Ingestion

The Nmap scan output was uploaded via Settings → Add Data → Upload, configured with a custom, reusable sourcetype and a meaningful host label representing the scan origin:

- Sourcetype: nmap_scan (custom)
- Host: kali-attacker
- Index: default (main)

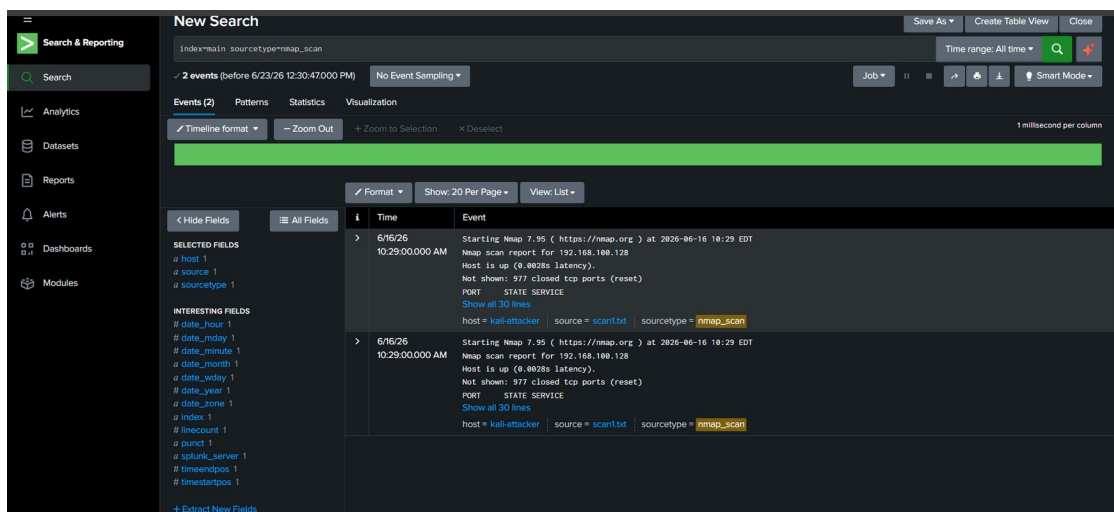


Figure 1. Raw Nmap events as ingested into Splunk, showing host, source, and sourcetype fields.

3.2 Baseline SPL Searches

Initial Search Processing Language (SPL) queries confirmed ingestion and demonstrated threat-hunting for known-malicious ports:

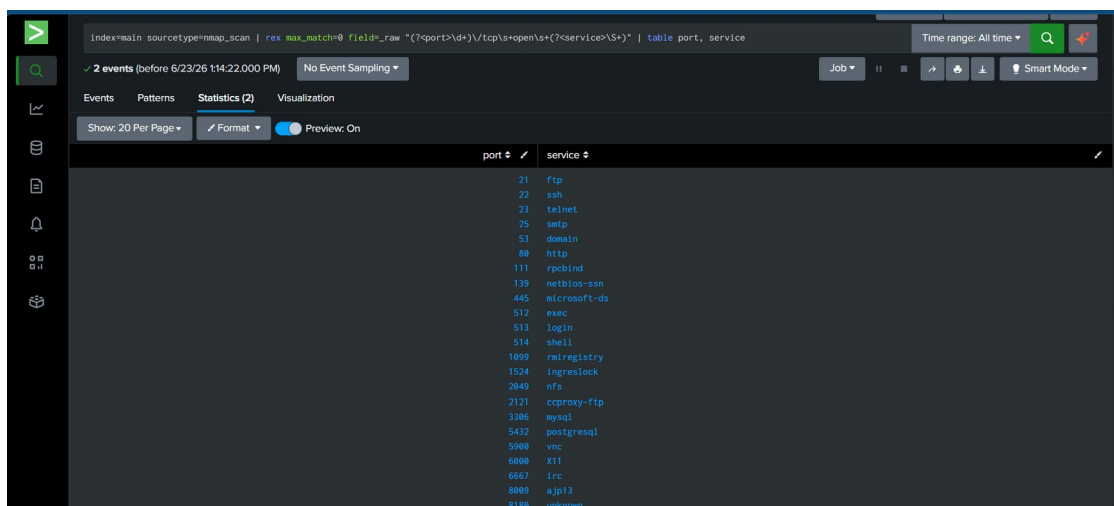
```
index=main host="kali-attacker" index=main sourcetype=nmap_scan "open" index=main
sourcetype=nmap_scan ("1524" OR "445" OR "21") index=main sourcetype=nmap_scan | stats
count
```

3.3 Field Extraction

Splunk's default extraction captures only the first regex match per event. Using `max_match=0` extracts every port/service pair from the raw scan text:

```
index=main sourcetype=nmap_scan | rex max_match=0 field=_raw "(?<port>\d+)\tcp\s+open\s+(?
<service>\S+)" | table port, service
```

This correctly extracted all 23 open ports (ftp, ssh, telnet, smtp, samba, mysql, postgresql, vnc, irc, ingreslock, and others).



The screenshot shows a Splunk search interface with the following search query: `index=main sourcetype=nmap_scan | rex max_match=0 field=_raw "(?<port>\d+)\tcp\s+open\s+(?<service>\S+)" | table port, service`. The results are displayed in a table with two columns: **port** and **service**.

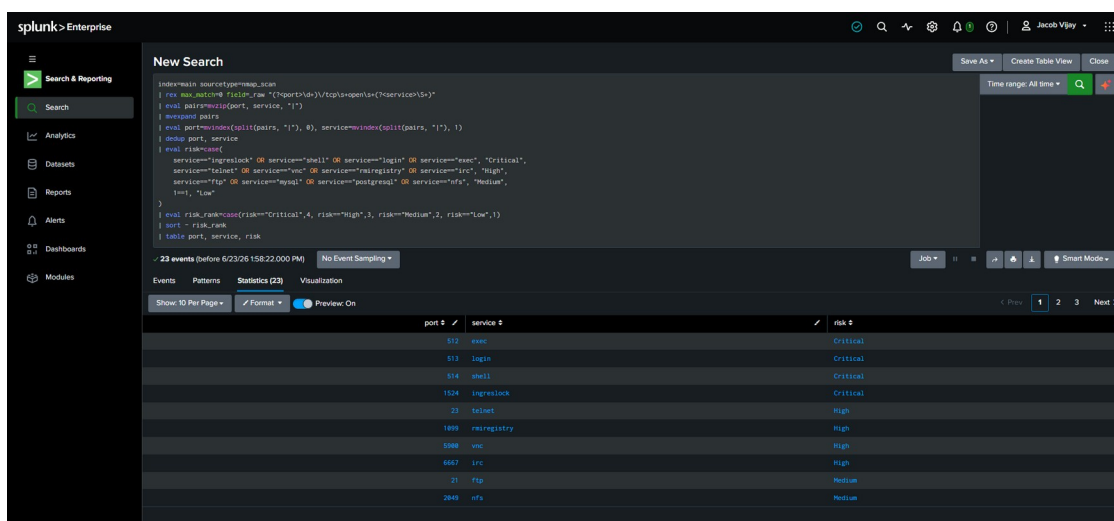
port	service
21	ftp
22	ssh
23	telnet
25	smtp
53	domain
80	http
111	rpcbind
139	netbios-ssn
445	microsoft-ds
512	exec
513	login
514	shell
1699	rmiregistry
1524	ingreslock
2049	nfs
2121	ccproxy-ftp
3306	mysql
5432	postgresql
5900	vnc
6000	X11
6667	irc
8080	ajp13
8100	unknown

Figure 2. Field extraction result showing all open ports and services parsed from the raw Nmap event.

3.4 Multivalue Expansion, Deduplication & Risk Classification

The extracted fields were multivalue (lists held within a single event), which prevented per-row logic. They were split into discrete rows, deduplicated to remove duplicate ingested events, then classified by risk severity:

```
index=main sourcetype=nmap_scan earliest=0 | rex max_match=0 field=_raw "(?<port>\d+)\tcp\s+open\s+(?<service>\S+)" | eval pairs=mvzip(port, service, "|") | mvexpand pairs | eval port=mvindex(split(pairs, "|"), 0), service=mvindex(split(pairs, "|"), 1) | dedup port, service | eval risk=case( service=="ingreslock" OR service=="shell" OR service=="login" OR service=="exec", "Critical", service=="telnet" OR service=="vnc" OR service=="rmiregistry" OR service=="irc", "High", service=="ftp" OR service=="mysql" OR service=="postgresql" OR service=="nfs", "Medium", 1==1, "Low") | eval risk_rank=case(risk=="Critical", 4, risk=="High", 3, risk=="Medium", 2, risk=="Low", 1) | sort - risk_rank | table port, service, risk
```



The screenshot shows a Splunk search interface with a more complex search query. The results are displayed in a table with three columns: **port**, **service**, and **risk**.

port	service	risk
512	exec	Critical
513	login	Critical
514	shell	Critical
1524	ingreslock	Critical
23	telnet	High
1699	rmiregistry	High
5900	vnc	High
6667	irc	High
21	ftp	Medium
2049	nfs	Medium

Figure 3. Risk-classified port inventory, sorted Critical → High → Medium.

4. Risk Classification Model

Each exposed service was assigned a severity tier based on real-world exploitability, informed directly by the penetration test against the same target.

Risk	Services	Rationale
Critical	ingreslock, shell, login, exec	Instant-root backdoors and unauthenticated remote command execution; exploited directly during the pentest
High	telnet, vnc, rmiregistry, irc	Legacy / weak protocols, cleartext credentials, known remote code execution vectors
Medium	ftp, mysql, postgresql, nfs	Commonly misconfigured services exposing data or weak authentication
Low	all remaining services	Standard services requiring deeper contextual review

5. SOC Dashboard

A Classic Dashboard titled "Metasploitable2 SOC Dashboard" was built with two statistics-table panels that together form a triage view:

- Open Ports — Metasploitable2 Scan: complete raw port/service inventory.
- Port Risk Classification: severity-tagged view, sorted Critical → High → Medium → Low for analyst triage.

DESIGN NOTE The two-panel layout mirrors a real SOC triage screen — a full asset inventory beside a prioritized severity view — so an analyst can move from "what is exposed" to "what must be addressed first" at a glance.

port	service
21	ftp
22	ssh
23	telnet
25	smtp
53	domain
80	http
111	rpcbind
139	netbios-ssn
445	microsoft-ds
512	exec
513	login
514	shell
1609	rmiregistry
1524	ingreslock
3306	mysql
5432	postgresql
5900	vnc
6880	x11
6881	irc
8080	ajp13
8188	unknown

Figure 4. Metasploitable2 SOC Dashboard — Open Ports panel.



port #	service #	risk #
512	exec	Critical
513	login	Critical
514	shell	Critical
1524	ingreslock	Critical
23	telnet	High
1099	rmiregistry	High
5900	vnc	High
6667	irc	High
21	ftp	Medium
2049	nfs	Medium

Figure 5. Metasploitable2 SOC Dashboard — Port Risk Classification panel.

6. Detection Alert

A scheduled alert named "Critical Backdoor Service Detected" was configured to monitor for instant-root backdoor services. The underlying search isolates only Critical-tier findings:

```
index=main sourcetype=nmap_scan earliest=0| rex max_match=0 field=_raw "(?<port>\d+)\tcp\
s+open\s+(?<service>\S+)"| eval pairs=mvizip(port, service, "|")| mvexpand pairs| eval
port=mvindex(split(pairs,"|"),0), service=mvindex(split(pairs,"|"),1)| dedup port, service| where
service IN ("ingreslock", "shell", "login", "exec")
```

Setting	Value
Schedule	Hourly (0 minutes past the hour)
Trigger Condition	Number of results is greater than 0
Trigger Action	Add to Triggered Alerts
Permissions	Private (owner: jacob)

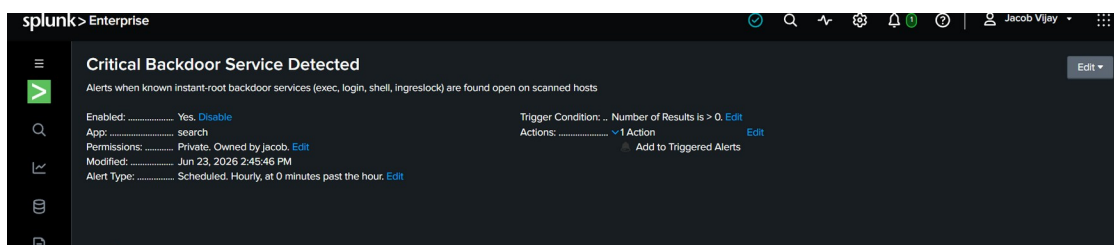


Figure 6. Saved alert configuration — Critical Backdoor Service Detected.

7. Lessons Learned

Several Splunk-specific behaviors were encountered and resolved during the build. These reflect real operational knowledge a SOC analyst is expected to hold.

- Splunk is append-only. Re-uploading a file duplicates rather than overwrites data; duplicates require | delete or dedup.

- delete is privilege-gated. Even the Administrator account lacks the delete_by_keyword capability by default — a deliberate least-privilege control that must be granted at the role level.
- Inline UI field extractions capture only the first match. Multivalue extraction needs max_match=0 in-search, or MV_ADD=true in transforms.conf.
- The time-range picker does not persist for scheduled alerts. Reliable scheduling requires hardcoding the range into the search text (earliest=0).
- Manually running an alert search does not fire its actions. Only the scheduler firing on the configured cron triggers the alert action.

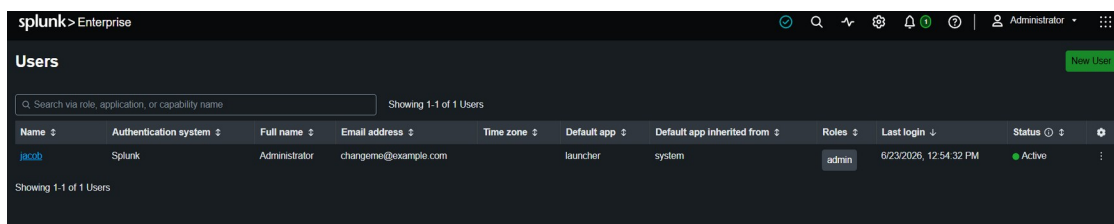


Figure 7. User-role assignment confirming the admin role before granting the delete_by_keyword capability.

8. Skills Demonstrated

Skill	Implementation
Data ingestion	Custom sourcetype nmap_scan, host kali-attacker
SPL	Field filtering, Boolean search, stats, keyword highlighting
Field extraction	rex with max_match=0 for full multivalue capture
Multivalue handling	mvzip + mvexpand + mvindex(split())
Deduplication	dedup to collapse duplicate events
Risk classification	eval case() severity tiering
Result ranking	rank field + sort for severity ordering
Dashboards	Classic dashboard, two statistics-table panels
Detection alerting	Scheduled alert with trigger + action
RBAC / capabilities	Granted delete_by_keyword at role level
Time-range scoping	earliest=0 for reliable scheduled search

9. Next Steps

- Ingest additional data sources (Metasploit exploit logs, DVWA access logs, Wireshark exports) for multi-source correlation.
- Add visualization panels such as a severity-distribution bar or pie chart.
- Build correlation searches that link reconnaissance, exploitation, and post-exploitation stages.
- Progress toward Splunk Core Certified User / Power User certification.

End of Report